

SHOPSPHERE – RETAIL DATA ANALYTICS

Retail Sales Analytics Using Excel, SQL, Power BI and Python

Table of Contents

1. Introduction
2. Problem Statement
3. Business Use Cases
4. Project Objectives
5. Tools and Technologies
6. Dataset Description
7. Project Approach
8. Phase 1 – Excel: Data Cleaning & Preparation
9. Phase 2 – SQL: Database Management & Querying
10. Phase 3 – Python & Machine Learning
11. Phase 4 – Power BI: Visualization & Dashboarding
12. Expected Results and Insights
13. Project Evaluation Metrics
14. Project Deliverables
15. Project Guidelines
16. Conclusion

1. Introduction

This project presents an end-to-end Retail Sales Analytics solution using Excel, SQL, Python, Machine Learning and Power BI. The project is designed to transform raw transactional data into structured data, analytical insights, customer segments and interactive management dashboards.

The overall workflow integrates data cleaning and preprocessing, relational database management, exploratory data analysis, customer segmentation and business intelligence reporting.

2. Problem Statement

In a competitive retail market, businesses need to track sales performance, understand customer behavior, and identify growth opportunities. This project aims to build an end-to-end analytics solution that enables insights generation from raw transactional data using a multi-tool approach.

3. Business Use Cases

1. Analyze sales performance across different stores and staff.
2. Identify high-value customers and buying behavior patterns.
3. Perform customer segmentation for targeted marketing.
4. Monitor inventory levels and product performance.
5. Create an interactive Power BI dashboard for management insights.

4. Project Objectives

- Clean and prepare retail data for analysis.
- Store and query structured data using SQL.
- Perform exploratory data analysis using Python.
- Calculate Recency, Frequency and Monetary (RFM) features.
- Create customer segments using clustering / KMeans where applicable.
- Build interactive Power BI dashboards.
- Integrate the complete ETL pipeline from Excel to Power BI.

5. Tools and Technologies

Tool / Technology	Purpose
Excel	Data cleaning and preprocessing
SQL	Data storage, querying and transformation
Python	EDA and analytical processing
Pandas / NumPy	Data manipulation and numerical analysis
Matplotlib / Seaborn	Data visualization
KMeans / Clustering	Customer segmentation
Power BI	Business intelligence and dashboarding

6. Dataset Description

Data Set: Retail Sales Data

Source: Provided schema diagram

Format: Relational Table Schema (to be exported to CSV/Excel and loaded to SQL)

The dataset consists of the following tables:

- customers
- orders
- order_items
- products
- brands
- categories
- stocks
- stores
- staffs

6.1 Table Explanation

Table	Description
customers	Buyer information including contact and location.
orders	Transaction log with staff and store references.
order_items	Detailed item information per order.
products	Item details with category and brand.
brands / categories	Metadata about items.
stocks	Inventory across stores.
stores / staffs	Organizational structure.

7. Project Approach

The project follows four major phases: Excel for data preparation, SQL for database management and querying, Python/ML for analytics and segmentation, and Power BI for visualization and reporting.

8. Excel: Data Cleaning & Preparation

Standardize Column Headers: Ensure columns across sheets are consistently named using conventions such as snake_case or camelCase.

Remove Duplicates: Identify and remove duplicate records in tables such as customers or order_items.

Handle Missing Values: Use filtering and formulas to find blanks/nulls and fill missing ZIP codes or phone numbers with placeholders or NA.

Data Type Conversion: Convert order dates, shipped dates and required dates into Excel Date format and ensure numeric fields are correctly formatted.

Data Validation: Use dropdowns for fields such as order_status to ensure consistency.

Create Derived Columns: Calculate total_price using the specified list price, quantity and discount information.

Merge Lookup Data: Use VLOOKUP or XLOOKUP to merge product names into order_items using product_id.

Create Basic Pivot Table: Summarize total sales by product category.

Sort and Filter for Outliers: Identify unusually high or low list_price values.

Prepare Final CSVs: Save cleaned sheets as CSV files such as customers.csv and orders.csv for SQL import.

9. SQL: Database Management & Querying

Create Tables Based on ERD: Use CREATE TABLE statements to replicate the structure of the ER diagram with constraints.

Import CSVs into SQL: Load cleaned Excel files using LOAD DATA or MySQL Workbench import.

Inner Join for Order Details: Join orders, order_items and products to display detailed line items.

Total Sales by Store: Group total_price by store_id.

Top 5 Selling Products: Use ORDER BY and LIMIT to identify the five most sold products by quantity.

Customer Purchase Summary: Return total orders, total items purchased and total revenue for each customer.

Segment Customers by Total Spend: Classify customers into spending brackets such as low, medium and high.

Staff Performance Analysis: Analyze total revenue generated by each staff member based on handled orders.

Stock Alert Query: List products where stock quantity is below 10 in any store.

Create Final Segmentation Table: Create customer_segments to receive segmentation results from Python ML.

10. Python & Machine Learning

Load Data from SQL: Use pandas.read_sql() to pull orders, order_items and customers into a DataFrame.

Basic EDA: Use df.describe(), df.info() and df.value_counts() to summarize the dataset and create basic charts.

Calculate RFM Features: Compute Recency, Frequency and Monetary values for each customer.

Recency: Days since the customer's last order.

Frequency: Number of orders.

Monetary: Total value of all purchases.

Export Segmentation Results to SQL: Save rfm_data with the segment label to the customer_segments table.

11. Power BI: Visualization & Dashboarding

Connect Power BI to SQL: Import key tables including orders, products, order_items, customers, stores and staffs.

Create Relationships: Define foreign-key relationships in Model View based on the ER diagram.

Sales Overview Report: Create visuals for total sales over time, monthly sales trend and total orders placed.

Top Products by Sales: Use a bar chart or table to list top-selling products by revenue or quantity.

Customer Purchase Analysis: Show purchase patterns by city or state using customers and orders.

Sales by Store Map: Use store.state or store.zip_code to show geographic sales distribution.

Low Stock Alert Dashboard: Use conditional formatting and cards for products below the stock threshold.

Interactive Filters and Slicers: Add slicers for Order Status, Product Category and Store.

Staff Performance Report: Show total sales/revenue per staff member.

Consolidated Dashboard Page: Include KPIs such as Total Revenue, Active Customers, Average Order Value and Total Orders.

Import customer_segments: Load segmentation results from SQL into Power BI.

Visualize Customer Segments: Use Pie Chart, Donut or TreeMap to show segment distribution.

Segment-Level Revenue Breakdown: Show total revenue per customer segment.

Segment Report Filters: Enable filtering by customer segment using slicers.

12. Expected Results and Insights

- Fully cleaned, relational dataset stored in SQL.
- Insightful analytics using Python.
- Segmentation-based insights using ML.
- Interactive dashboard in Power BI.
- Complete ETL pipeline from Excel to Power BI.

13. Project Evaluation Metrics

- Accuracy of cleaned and processed data.
- SQL query proficiency, including joins, aggregations and filters.
- Quality of Python EDA and ML models, if used.
- Insight depth and relevance.
- Power BI dashboard interactivity and design.
- Code structure, comments and modularity.

14. Project Deliverables

- Cleaned Excel files.
- SQL database creation scripts and query scripts.
- Python notebook or script with analysis and ML.
- Power BI .pbix file with visuals.
- Final report documenting steps, insights and visuals.

15. Project Guidelines

- Use proper naming conventions and comments in SQL/Python.
- Keep backups of Excel and SQL stages.
- Use GitHub or local versioning if collaborative.
- Follow visualization best practices: label axes, use tooltips and limit clutter.
- Submit documentation in PDF or markdown format.

16. Conclusion

The Retail Sales Analytics project provides a structured end-to-end approach for turning retail transactional data into business intelligence. By combining Excel, SQL, Python, Machine Learning and Power BI, the workflow covers data preparation, storage, analysis, customer segmentation, inventory monitoring and interactive reporting.

The project is intended to support management insights through sales performance analysis, customer behavior analysis, product and inventory monitoring, staff performance reporting and customer segment visualization.